CredChain

Build Guide & Run Manual

Global Trust Edition - Nigeria Launch. A decentralized, high-trust credential identity network on Solana. Trust infrastructure that doesn't care how famous your school is.

Prepared: 21 June 2026

Stack: React + Vite + Tailwind (3000) - Node/Express + MongoDB + Socket.io + Solana (5000)
FastAPI CV engine 'Tony' (8001) - FastAPI Insights 'Zhavia' (8002)

---

1. What CredChain Is

CredChain gives people with real, demonstrable skill a fair shot at the global job market - regardless of which country they are from or whether their school, bootcamp, or community is internationally famous. It replaces reputation-by-fame with cryptographic proof: once a real, accountable issuer vouches for an achievement, a SHA-256 fingerprint of it is anchored on the Solana Devnet and is equally verifiable anywhere.

It launches on Nigerian campuses first (transcript delays of 6-12 months, NYSC mobilization errors, hiring bias) and is architected from day one to expand to any country via a per-country 'Country Module'.

The non-negotiables baked into the build

- Anti-fraud backbone: students never self-certify onto the Verified Ledger - every verified credential is vouched for by a verified Issuer.
- Radically inclusive issuer onboarding: legitimacy is judged by 'is this a real, accountable entity', never by fame. Any country, any size has a real path in.
- No crypto friction: the platform pays gas server-side (custodial fee-payer). No user ever sees a wallet, gas fee, or seed phrase.
- Fairness rule: the CredScore and all ranking logic take ONLY verifiable evidence as input - never country, institution prestige, or any wealth-correlated proxy.

2. Architecture (4 services, isolated ports)

Port	Service	Tech	Responsibility
3000	Frontend	React, Vite, Tailwind, React Router, Axios, socket.io-client	The three portals + public pages
5000	Backend Core	Node.js, Express, MongoDB, Socket.io, @solana/web3.js	Data gateway, JWT, OAuth, on-chain anchoring
8001	AI CV Engine (Tony)	Python, FastAPI, fpdf2	Generates verified, print-ready PDF resumes
8002	AI Insights (Zhavia)	Python, FastAPI	Market telemetry / skills-gap (currency-aware)

On-chain anchoring uses the SPL Memo program on Solana Devnet - only the 64-char hash is written, never personal data. If no fee-payer wallet is configured the app still works and records credentials off-chain (the fingerprint stays tamper-evident).

3. What Was Built

The backend's advanced /api/v1 layer existed already; this work added Google OAuth, the dispute system, the employer talent feed, a chat-rooms list, and an admin issuer-list endpoint - then built the ENTIRE frontend (it was previously a skeleton).

Authentication (Google OAuth + RBAC)

- Real Google OAuth 2.0 added to the backend (no passport - a direct code-exchange flow). GET /api/v1/auth/google?role= -> Google -> callback -> /auth/callback?token=&role=.
- AuthCallback.jsx stores the JWT, updates a global AuthContext, sets the Axios auth header, and routes to the correct portal.
- Email/password login is kept as a working fallback. ProtectedRoute enforces role-based access across /student, /employer, /issuer.

Portal A - Student Vault (/student)

- Two-Step Approval Queue: Accept (anchors on Solana) / Reject pending credentials - live.
- Two-Tier Ledger: Verified (issuer-anchored) vs Sandbox (self-taught) - honestly labelled.
- Animated CredScore gauge (300-850), computed from evidence only (no bias inputs).
- AI Co-Pilot: Generate Verified CV (downloads a real PDF from :8001) + Sync Market Telemetry (:8002).
- View On-Chain Proof modal (W3C Verifiable Credential JSON + fingerprint + Explorer link).
- Live SVG trust badges, Share/Export drawer (public link, QR, one-click LinkedIn export).
- Full lifecycle audit trail per credential, and a Messages inbox to reply to recruiters.
- Nigeria module: Instant Statement of Result, NYSC Pre-Validation tracker, Global Trust Pass (national-ID hash, raw ID never stored), Low-Data Offline Pass / USSD shortcode.

Portal B - Employer Suite (/employer)

- Trust-First Talent Feed from REAL students, with a 'Hide Unverified Skills' strict-mode toggle.
- 'Who Issued This?' chips link to the Public Issuer Registry.
- Token-bucket anti-spam chat (REAL): 50 credits, -1 to open a chat, refunded the moment the student replies; rooms show LOCKED until then; context credential auto-pinned; live over Socket.io.
- Employer-Sponsored Micro-Bounties board (passed test suite -> auto-mint a verified credential).
- Verification Report export (JSON + printable PDF) - the proof-of-due-diligence artefact.
- Outreach outcome stats (messaged / replied).

Portal C - Issuer Command Center (/issuer)

- Polymorphic onboarding wizard, Types A-E (university / bootcamp / event / OSS-DAO / professional body). Fields change by type and Country Module; Nigeria is fully automated, other countries gracefully fall back to manual review. Maps onto the real funnel: register -> live DNS TXT verification -> KYC -> awaiting admin Tier-4 vetting.
- Issue a single credential + Proof-of-Skill auto-issuer (webhook endpoints + simulate winner).
- Enterprise Bulk-Upload: drag-drop CSV, Maker-Checker dual approval, live 0-100% progress bar streamed over Socket.io.
- On-Chain Revocation Registry (appends :REVOKED on Solana).
- Issuer Reputation Dashboard (placement rate, avg CredScore, time-to-hire).

Cross-portal + trust infrastructure

- Dispute & Appeal flow (REAL): a student disputes a revocation -> badge freezes to amber 'Under Review' -> routed to an INDEPENDENT platform-admin queue (never back to the issuer).
- Public Issuer Registry (/registry) and Equity Impact Dashboard (/impact) - logged-out, browsable.
- Public student verification page (/verify/student/:credchainId) - real profile + live badges.
- Platform Admin Panel (/admin): Overview, Issuer Vetting (the Tier-4 cross-match that unlocks issuance), and the Dispute queue. Gated to an ADMIN_EMAILS allowlist.

4. Key Files (what to open if you want to read the code)

Backend (backend/src)

controllers/authController.js	Google OAuth code-exchange flow
controllers/credentialController.js	issue, revoke, dispute, admin list/resolve disputes
controllers/chatController.js	token-bucket chat + listRooms
controllers/employerController.js	talent feed + chat credits
controllers/issuerController.js	the funnel + admin issuer list
controllers/badgeController.js	live SVG badge (green / amber / red)
routes/v1.js	all /api/v1 routes (the contract)
models/*.js	User, Credential (+dispute), IssuerProfile, ChatRoom, EmployerProfile, StudentProfile

Frontend (frontend/src)

App.jsx / main.jsx	router + RBAC + AuthProvider
context/AuthContext.jsx	session, role, Axios header
services/api.js	every Axios call to the backend
portals/	StudentPortal, EmployerPortal, IssuerPortal, AdminPanel, public/*
components/student/*	queue, ledger, CredScore gauge, proof modal, inbox, nigeria/*
components/issuer/*	onboarding wizard, bulk upload, revocation, reputation
components/employer/*	talent feed, chat drawer, micro-bounties
components/admin/*	overview, issuer vetting, dispute queue
config/countryModules.js	per-country verification config (NG fully wired)
lib/credScore.js	fairness-safe scoring (evidence only)

5. Prerequisites

- Node.js 18+ and npm.
- Python 3 (the two AI engines already have a venv with fpdf2 + openai installed under ai-cv-engine/venv and ai-insights-engine/venv).
- MongoDB - the backend .env already points at a MongoDB Atlas cluster, so no local install needed.
- A Google Cloud OAuth client (only if you want the 'Sign in with Google' button to work; otherwise use the email/password fallback).

6. Environment Setup

The backend reads backend/.env. The important values are already set (JWT secret, MongoDB Atlas URI, Solana RPC). To unlock the optional features, fill these in:

Google sign-in (optional)

Create an OAuth client at console.cloud.google.com -> APIs & Services -> Credentials. The Authorized redirect URI must be exactly the callback URL below. Then in backend/.env:

```
GOOGLE_CLIENT_ID=your-id.apps.googleusercontent.com
GOOGLE_CLIENT_SECRET=your-secret
GOOGLE_CALLBACK_URL=http://localhost:5000/api/v1/auth/google/callback
```

Until these are real, the Google button cleanly shows 'not configured' and you use email/password instead.

Admin panel access (needed to vet issuers / resolve disputes)

Add the email of the account you will log in with to the admin allowlist in backend/.env (comma-separated for several). Restart the backend after editing.

```
ADMIN_EMAILS=you@example.com
```

On-chain writes (optional)

Without a funded Devnet wallet, Accept/Revoke still work but skip the Solana write. To write real transactions, set SOLANA_WALLET_PATH in backend/.env to a funded Devnet wallet JSON (array of secret-key bytes).

7. How to Run Everything

Option A - one command (recommended)

From the project root (C:\Chris Stuff\CredChain). First time only, install dependencies:

```
npm install                # root tools (concurrently)
npm run install:all        # installs backend + frontend node deps
# Python deps are already installed in the two venvs.
```

Then start all four services together (root package.json uses 'concurrently'):

```
npm run dev
```

This launches: backend (5000), frontend (3000), CV engine (8001), Insights (8002), each colour-labelled in the terminal. Open http://localhost:3000.

Option B - start each service in its own terminal

```
# Terminal 1 - backend
cd backend && npm run dev

# Terminal 2 - frontend
cd frontend && npm run dev

# Terminal 3 - AI CV engine (Tony)
cd ai-cv-engine
venv\Scripts\python.exe -m uvicorn src.api:app --port 8001 --reload

# Terminal 4 - AI Insights (Zhavia)
cd ai-insights-engine
venv\Scripts\python.exe -m uvicorn src.api:app --port 8002 --reload
```

Note: the frontend is the only piece you strictly need for the UI. The AI engines are only required for 'Generate Verified CV' and 'Sync Market Telemetry'; everything else works with just backend + frontend.

8. End-to-End Demo Walkthrough

This is the chain that shows the whole trust model working. Open http://localhost:3000.

Set up accounts

- Register three accounts (email/password is fine): one student, one employer, one issuer. Use the role toggle on /register.
- Put the issuer's email (or your own) in ADMIN_EMAILS so you can also act as platform admin.

Issuer gets verified

- Log in as the issuer -> /issuer -> 'Get Verified'. Pick a Type and country, submit. For an automated country (Nigeria) it gives a real DNS TXT challenge; for others it routes to manual review. Complete the steps to reach 'awaiting admin vetting'.
- Log in as the admin -> /admin -> Issuer Vetting -> Approve. This runs the Tier-4 cross-match and flips the issuer to active (issuance unlocked).

Issue -> Accept -> Prove

- As the verified issuer -> 'Issue & Auto-Issue' -> issue a credential to the student's email (or try Bulk Upload with Maker-Checker).
- Log in as the student -> /student -> the credential is in the Pending Queue -> Accept (it anchors). Open 'View On-Chain Proof' to show the fingerprint + Explorer link. Watch the CredScore gauge move.

Employer sources + chats

- Log in as the employer -> /employer -> the student appears in the Trust-First feed. Toggle 'Hide Unverified'. Click Message (spends 1 of 50 credits; room shows LOCKED).
- Log back in as the student -> Messages inbox -> reply. The room unlocks and the employer's credit is refunded (visible live).
- Export a Verification Report (PDF/JSON) from the employer feed.

Revoke -> Dispute -> Resolve

- As the issuer -> Revocation -> revoke the credential (badge goes red).
- As the student -> the revoked card shows 'Dispute this revocation' -> file it. The badge turns amber 'Under Review'.
- As the admin -> /admin -> Disputes -> Reinstate (badge returns to green) or Uphold (stays red). This is the independent check that answers 'what stops an issuer abusing this?'.

Public pages (no login)

- /registry - Public Issuer Registry and trust tiers.
- /impact - Equity Impact Dashboard.
- /verify/student/<credchainId> - anyone can verify a student's on-chain credentials.

9. Important Notes

- Google OAuth and the Admin panel both need config (Section 6) before they work; everything else runs out of the box.
- The employer talent feed only shows students who have at least one ACCEPTED credential - so issue and accept one before demoing the employer side.
- Mock data is used (clearly) for the Public Registry, Equity Dashboard, Micro-Bounties and the Issuer Reputation Dashboard - these are presentation surfaces with no aggregation backend yet.
- The Issuer wizard's type-specific registry fields (OPEID, CAC, etc.) are collected on the frontend; only the core funnel (domain/DNS/KYC) is persisted server-side.
- Solana writes are skipped unless a funded Devnet fee-payer wallet is configured - credentials still record off-chain and stay tamper-evident.

10. Troubleshooting

Port already in use	Something else is on 3000/5000/8001/8002 - stop it or change the port.
'Admins only' on /admin	Add your login email to ADMIN_EMAILS in backend/.env and restart backend.
Google button errors	GOOGLE_CLIENT_ID/SECRET not set, or redirect URI mismatch in Google Console.
CV / telemetry buttons fail	Start the AI engines (8001/8002); they are optional for the rest.
Empty employer feed	No students with accepted credentials yet - issue + accept one first.
Mongo connection warning	Backend logs it and keeps running; check the MONGO_URI in backend/.env.
Badge shows red unexpectedly	It actively checks integrity live; a disputed revocation shows amber, not red.

The pitch in one line: trust infrastructure that doesn't care how famous your school is. A bootcamp in Lagos and a bootcamp in Manila go through the exact same trust framework.